

LAPORAN PRAKTIKUM DAN TUGAS 2
SISTEM OPERASI



Dosen pengampu :
Triawan Adi Cahyono M.Kom

Disusun Oleh:
Nama : Putri Suci Rahmadhani
Nim : 2410651008
Tanggal : 17 Oktober 2025

PRODI TEKNIK INFORMATIKA FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

1.1. TUJUAN PERCOBAAN DAN TUGAS

Tujuan dari seluruh percobaan dan tugas ini adalah untuk memahami konsep dasar proses dalam sistem operasi Linux, khususnya bagaimana proses dibuat, dijalankan, dan berinteraksi satu sama lain menggunakan bahasa C di lingkungan Ubuntu. Melalui percobaan 1.1 hingga 1.4, mahasiswa mempelajari pembuatan proses baru dengan fungsi `fork()`, pengaturan urutan eksekusi menggunakan `wait()`, serta penggunaan fungsi `exec()` untuk mengganti program yang sedang berjalan. Sedangkan pada percobaan 2.1 hingga 2.4, mahasiswa mempelajari berbagai metode komunikasi antar proses (Inter Process Communication/IPC) seperti pipe, FIFO (named pipe), shared memory, dan message queue, untuk memahami cara pertukaran data antar proses yang berjalan secara bersamaan.

Selain itu, pada Tugas 1, mahasiswa dilatih untuk memahami konsep dasar proses dalam sistem operasi, membuat proses baru dengan `fork()`, mengamati hubungan antara proses induk (parent) dan anak (child), serta melatih penggunaan perintah dasar terminal Ubuntu seperti `gcc`. Sedangkan Tugas 2 bertujuan agar mahasiswa dapat mempelajari cara dua proses berbeda saling berkomunikasi, mengenal konsep IPC dengan FIFO, melihat bagaimana dua terminal dapat bertukar pesan langsung, serta menerapkan sinkronisasi antar proses di Ubuntu.

Jadi kegiatan ini bertujuan untuk melatih pemahaman tentang mekanisme proses dan komunikasi antar proses di sistem operasi linux, baik secara teori maupun implementasi praktis menggunakan Bahasa c.

1.2.DASAR TEORI

1. Konsep proses

Dalam tugas 1 digunakan fungsi `fork()`; yang artinya untuk membuat proses baru(child proses) dari proses yang sedang berjalan (parent proses). Setelah `fork()` dijalankan, maka akan ada 2 proses aktif yaitu :

- a. Parent process : proses utama yang memanggil `fork()`
- b. Child process : proses baru hasil Salinan dari parent.

Kedua proses ini kemudian dapat menjalankan intruksi yang berbeda. Tetapi secara bersamaan . hal ini menunjukkan bagaimana sistem operasi mengatur dan membagi waktu CPU di antara beberapa proses.

2. Ipc method

Dalam tugas 2 konsep yang digunakan Adalah inter-process communication(IPC) yaitu mekanisme pertukaran data antar dua proses yang berjalan secara bersamaan. Salah satu metode IPC yang digunakan di tugas 2 adalah FIFO (named pipe). Yaitu “`mkfifo(“nama_fifo”,0666);`” setiap proses bisa menulis ke FIFO menggunakan `read()`. FIFO bekerja berdasarkan prinsip first in first out,

artinya data yang pertama dikirim akan menjadi data pertama yang diterima. Dalam percobaan tugas 2, dua terminal digunakan untuk mewakili dua proses.

- a. **User 1** menulis pesan ke chat_user1 dan membaca dari chat_user2.
 - b. **User 2** menulis pesan ke chat_user2 dan membaca dari chat_user1.
- Dengan konsep ini, dua proses dapat berkomunikasi secara langsung.

1.3.PERCobaAN

1. Percobaan 1.1

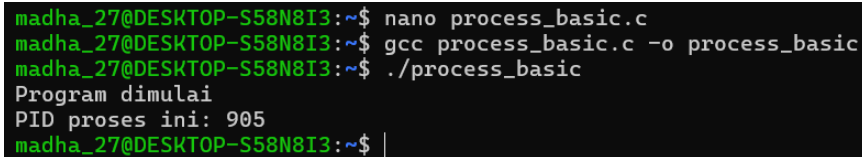
a. Codingan

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;
    printf("Program dimulai\n");
    printf("PID proses ini: %d\n", getpid());

    return 0;
}
```

b. Hasil output program



```
madha_27@DESKTOP-S58N8I3:~$ nano process_basic.c
madha_27@DESKTOP-S58N8I3:~$ gcc process_basic.c -o process_basic
madha_27@DESKTOP-S58N8I3:~$ ./process_basic
Program dimulai
PID proses ini: 905
madha_27@DESKTOP-S58N8I3:~$ |
```

c. Analisis

Pada percobaan ini kita belajar cara menampilkan ID proses (PID). Hasilnya menunjukkan bahwa setiap program yang dijalankan di sistem memiliki nomor proses yang berbeda. Ini membuktikan kalau sistem operasi bisa mengenali dan mengatur setiap proses secara terpisah.

2. Percobaan 1.2

a. Codingan

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;
    printf("Sebelum fork()\n");
    printf("PID: %d\n\n", getpid());
```

```

pid = fork();
if (pid < 0) {
    fprintf(stderr, "Fork gagal!\n");
    return 1;
}
else if (pid == 0) {
    // Child process
    printf("CHILD PROCESS\n");
    printf("PID saya: %d\n", getpid());
    printf("PID parent saya: %d\n", getppid());
}
else {
    // Parent process
    printf("PARENT PROCESS\n");
    printf("PID saya: %d\n", getpid());
    printf("PID child saya: %d\n", pid);
}
printf("Proses %d selesai\n", getpid());
return 0;
}

```

b. Hasil output program

```

madha_27@DESKTOP-S58N8I3:~$ nano fork_demo.c
madha_27@DESKTOP-S58N8I3:~$ gcc fork_demo.c -o fork_demo
madha_27@DESKTOP-S58N8I3:~$ ./fork_demo
Sebelum fork()
PID: 922

PARENT PROCESS
PID saya: 922
CHILD PROCESS
PID child saya: 923
Proses 922 selesai
PID saya: 923
PID parent saya: 922
Proses 923 selesai
madha_27@DESKTOP-S58N8I3:~$ |

```

c. Analisis

Di percobaan ini digunakan fungsi fork() untuk membuat proses baru. Setelah dijalankan, muncul dua proses: parent (induk) dan child (anak). Keduanya bisa berjalan bersamaan dan punya PID berbeda. Ini menunjukkan bahwa satu program bisa menurunkan proses lain.

3. Percobaan 1.3

a. Codingan

```

#include <stdio.h>
#include <unistd.h>

```

```

#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;
    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork gagal!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
        sleep(3);
        printf("Child: Selesai!\n");
        return 42; // Exit code
    }
    else {
        // Parent process
        printf("Parent: Menunggu child (PID: %d) selesai...\n", pid);
        wait(&status);

        if (WIFEXITED(status)) {
            printf("Parent: Child selesai dengan exit code: %d\n",
                WEXITSTATUS(status));
        }
    }
    return 0;
}

```

b. Hasil output program

```

madha_27@DESKTOP-S58N8I3:~$ nano process_wait.c
madha_27@DESKTOP-S58N8I3:~$ gcc process_wait.c -o process_wait
madha_27@DESKTOP-S58N8I3:~$ ./process_wait
Parent: Menunggu child (PID: 934) selesai...
Child: PID saya 934, akan tidur 3 detik
Child: Selesai!
Parent: Child selesai dengan exit code: 42
madha_27@DESKTOP-S58N8I3:~$ |

```

c. Analisis

Percobaan ini memperlihatkan fungsi wait() yang membuat proses induk menunggu sampai proses anak selesai. Hasilnya, proses jadi

berjalan lebih teratur dan tidak tumpang tindih. Ini penting supaya program berjalan dengan urutan yang benar.

4. Percobaan 1.4

a. Codingan

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid;
    int num_children = 3;
    printf("Parent PID: %d\n\n", getpid());

    // Membuat beberapa child process
    for (i = 0; i < num_children; i++) {
        pid = fork();

        if (pid < 0) {
            fprintf(stderr, "Fork gagal!\n");
            return 1;
        }
        else if (pid == 0) {
            // Child process
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());
            sleep(i + 1); // simulasi pekerjaan
            printf("Child %d selesai\n", i + 1);
            return 0; // keluar dari child process
        }
    }

    // Parent menunggu semua child selesai
    for (i = 0; i < num_children; i++) {
        int status;
        pid_t child_pid = wait(&status);
        printf("Parent: Child dengan PID %d selesai\n", child_pid);
    }

    printf("\nParent: Semua child selesai\n");
}
```

```
    return 0;
}
```

b. Hasil output program

```
madha_27@DESKTOP-S58N8I3:~$ nano multiple_fork.c
madha_27@DESKTOP-S58N8I3:~$ gcc multiple_fork.c -o multiple_fork
madha_27@DESKTOP-S58N8I3:~$ ./multiple_fork
Parent PID: 941

Child 1: PID = 942, Parent PID = 941
Child 2: PID = 943, Parent PID = 941
Child 3: PID = 944, Parent PID = 941
Child 1 selesai
Parent: Child dengan PID 942 selesai
Child 2 selesai
Parent: Child dengan PID 943 selesai
Child 3 selesai
Parent: Child dengan PID 944 selesai

Parent: Semua child selesai
madha_27@DESKTOP-S58N8I3:~$ |
```

c. Analisis

Fungsi `exec()` digunakan untuk mengganti program yang sedang berjalan dengan program lain. Jadi setelah dijalankan, proses lama berhenti dan diganti oleh program baru. Ini menunjukkan bagaimana sistem bisa menukar isi proses tanpa membuat proses baru.

5. Percobaan 2.1

a. Codingan

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
```

```
int main() {
    int pipefd[2];          // pipefd[0] = read, pipefd[1] = write
    pid_t pid;
    char write_msg[] = "Hello dari Parent!";
    char read_msg[100];

    // Membuat pipe
    if (pipe(pipefd) == -1) {
        perror("Pipe gagal");
        return 1;
    }

    pid = fork();
    if (pid < 0) {
        perror("Fork gagal");
        return 1;
    }
}
```

```

    }
    else if (pid == 0) {
        // Child process: membaca dari pipe
        close(pipefd[1]); // Tutup sisi write
        read(pipefd[0], read_msg, sizeof(read_msg));
        printf("Child menerima: %s\n", read_msg);
        close(pipefd[0]);
    }
    else {
        // Parent process: menulis ke pipe
        close(pipefd[0]); // Tutup sisi read
        printf("Parent mengirim: %s\n", write_msg);
        write(pipefd[1], write_msg, strlen(write_msg) + 1);
        close(pipefd[1]);
        wait(NULL); // Tunggu child selesai
    }

    return 0;
}

```

b. Hasil output program

```

madha_27@DESKTOP-S58N8I3:~$ nano pipe_demo.c
madha_27@DESKTOP-S58N8I3:~$ gcc pipe_demo.c -o pipe_demo
madha_27@DESKTOP-S58N8I3:~$ ./pipe_demo
Parent mengirim: Hello dari Parent!
Child menerima: Hello dari Parent!
madha_27@DESKTOP-S58N8I3:~$ |

```

c. Analisis

Percobaan ini menunjukkan cara dua proses saling mengirim data lewat pipe. Satu proses mengirim, dan proses lain menerima. Hasilnya, data bisa dikirim dengan urutan yang sama (FIFO). Artinya, pipe bisa digunakan untuk komunikasi antar proses di satu program.

6. Percobaan 2.2

a. Codingan

- Fifo_write


```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define SHM_SIZE 1024

int main() {

```



```

key_t key;
int shmid;
char *shared_memory;
char message[100];

// Generate unique key
key = ftok("shmfile", 65);

// Create shared memory segment
shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
if (shmid < 0) {
    perror("shmget gagal");
    exit(1);
}

// Attach ke shared memory
shared_memory = (char *) shmat(shmid, NULL, 0);
printf("Writer: Masukkan pesan: ");
fgets(message, sizeof(message), stdin);

// Tulis ke shared memory
strcpy(shared_memory, message);
printf("Writer: Data ditulis ke shared memory\n");

// Detach dari shared memory
shmdt(shared_memory);
return 0;
}

```

- Fifo_reader

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#define FIFO_FILE "/tmp/my_fifo"

int main() {
    int fd;
    char buffer[100];
    printf("Reader: Membuka FIFO...\n");
}

```

```

fd = open(FIFO_FILE, O_RDONLY);

read(fd, buffer, sizeof(buffer));
printf("Reader: Menerima pesan: %s\n", buffer);
close(fd);
unlink(FIFO_FILE); // Hapus FIFO setelah digunakan
return 0;
}

```

b. Hasil output program

- Fifo_write

```

madha_27@DESKTOP-S58N8I3:~$ nano fifo_write.c
madha_27@DESKTOP-S58N8I3:~$ gcc fifo_write.c -o fifo_write
madha_27@DESKTOP-S58N8I3:~$ ./fifo_write
Writer: Menunggu reader...
Masukkan pesan: halo fifo_reader
Writer: Pesan terkirim!
madha_27@DESKTOP-S58N8I3:~$ |

```

- Fifo_reader

```

madha_27@DESKTOP-S58N8I3:~$ nano fifo_reader.c
madha_27@DESKTOP-S58N8I3:~$ gcc fifo_reader.c -o fifo_reader
madha_27@DESKTOP-S58N8I3:~$ ./fifo_reader
Reader: Membuka FIFO...
Reader: Menerima pesan: halo fifo_reader

madha_27@DESKTOP-S58N8I3:~$ |

```

c. Analisis

FIFO hampir sama seperti pipe, tapi bisa digunakan antar dua program atau dua terminal berbeda. Proses writer mengirim pesan, dan reader menerima lewat file FIFO. Hasilnya, dua program yang terpisah bisa saling bertukar data secara langsung.

7. Percobaan 2.3

a. Codingan

- Shm_write


```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define SHM_SIZE 1024

int main() {
    key_t key;
    int shmid;

```

```

char *shared_memory;
char message[100];

// Generate unique key
key = ftok("shmfile", 65);

// Create shared memory segment
shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
if (shmid < 0) {
    perror("shmget gagal");
    exit(1);
}

// Attach ke shared memory
shared_memory = (char *) shmat(shmid, NULL, 0);

printf("Writer: Masukkan pesan: ");
fgets(message, sizeof(message), stdin);

// Tulis ke shared memory
strcpy(shared_memory, message);
printf("Writer: Data ditulis ke shared memory\n");

// Detach dari shared memory
shmdt(shared_memory);
return 0;
}

```

- Shm_reader

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define SHM_SIZE 1024

```

```

int main() {
    key_t key;
    int shmid;
    char *shared_memory;

    // Generate same key

```

```

key = ftok("shmfile", 65);

// Access shared memory yang sudah dibuat oleh writer
shmid = shmget(key, SHM_SIZE, 0666);
if (shmid < 0) {
    perror("shmget gagal");
    exit(1);
}

// Attach ke shared memory
shared_memory = (char *) shmat(shmid, NULL, 0);
printf("Reader: Data dari shared memory: %s\n",
shared_memory);

// Detach dan hapus shared memory
shmdt(shared_memory);
shmctl(shmid, IPC_RMID, NULL);

return 0;
}

```

b. Hasil output program

- Shm_write

```

madha_27@DESKTOP-S58N8I3:~$ nano shm_write.c
madha_27@DESKTOP-S58N8I3:~$ gcc shm_write.c -o shm_write
madha_27@DESKTOP-S58N8I3:~$ ./shm_write
Writer: Masukkan pesan: haiii
Writer: Data ditulis ke shared memory
madha_27@DESKTOP-S58N8I3:~$

```

- Shm_reader

```

madha_27@DESKTOP-S58N8I3:~$ nano shm_reader.c
madha_27@DESKTOP-S58N8I3:~$ gcc shm_reader.c -o shm_reader
madha_27@DESKTOP-S58N8I3:~$ ./shm_reader
Reader: Data dari shared memory: haiii
madha_27@DESKTOP-S58N8I3:~$ |

```

c. Analisis

Percobaan ini memakai shared memory, yaitu cara dua proses berbagi area memori yang sama. Hasilnya, data bisa diakses lebih cepat tanpa lewat file. Tapi perlu hati-hati supaya tidak ada proses yang menimpa data proses lain.

8. Percobaan 2.4

a. Codingan

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/time.h>
#include <sys/wait.h>

#define DATA_SIZE 10000
#define SHM_SIZE DATA_SIZE * sizeof(int)

// Fungsi untuk mendapatkan waktu saat ini (dalam detik)
double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

// Percobaan 1: Menggunakan PIPE
void test_pipe() {
    int pipefd[2];
    pid_t pid;
    int *data = malloc(DATA_SIZE * sizeof(int));
    double start, end;

    pipe(pipefd);
    pid = fork();
    if (pid == 0) { // Child Process (membaca data)
        close(pipefd[1]); // Tutup write-end
        read(pipefd[0], data, DATA_SIZE * sizeof(int));
        close(pipefd[0]);
        free(data);
        exit(0);
    } else { // Parent Process (menulis data)
        for (int i = 0; i < DATA_SIZE; i++) {
            data[i] = i;
        }

        start = get_time();

```

```

        write(pipefd[1], data, DATA_SIZE * sizeof(int));
        close(pipefd[1]);
        wait(NULL);
        end = get_time();
        printf("Pipe: %.6f detik\n", end - start);
        free(data);
    }
}

```

// Percobaan 2: Menggunakan SHARED MEMORY

```

void test_shared_memory() {
    key_t key = ftok("/tmp", 65);
    int shmid;
    int *shared_data;
    pid_t pid;
    double start, end;

    shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    start = get_time();

    pid = fork();

    if (pid == 0) { // Child Process (membaca data)
        shared_data = (int *)shmat(shmid, NULL, 0);
        long long sum = 0;
        for (int i = 0; i < DATA_SIZE; i++) {
            sum += shared_data[i];
        }
        shmdt(shared_data);
        exit(0);
    } else { // Parent Process (menulis data)
        shared_data = (int *)shmat(shmid, NULL, 0);
        for (int i = 0; i < DATA_SIZE; i++) {
            shared_data[i] = i;
        }

        wait(NULL);
        end = get_time();
        printf("Shared Memory: %.6f detik\n", end - start);
    }
}

```

```

        shmdt(shared_data);
        shmctl(shmid, IPC_RMID, NULL);
    }
}

// Fungsi Utama
int main() {
    printf("Benchmarking IPC Methods dengan %d integers\n\n",
DATA_SIZE);
    test_pipe();
    test_shared_memory();
    return 0;
}

```

b. Hasil output program

```

madha_27@DESKTOP-S58N8I3:~$ nano ipc_benchmark.c
madha_27@DESKTOP-S58N8I3:~$ gcc ipc_benchmark.c -o ipc_benchmark
madha_27@DESKTOP-S58N8I3:~$ ./ipc_benchmark
Benchmarking IPC Methods dengan 10000 integers

Pipe: 0.000389 detik
Shared Memory: 0.001076 detik
madha_27@DESKTOP-S58N8I3:~$ |

```

c. Analisis

Message queue digunakan untuk mengirim pesan antar proses lewat sistem antrian. Hasil percobaan menunjukkan pesan bisa dikirim dan diterima dengan urutan yang jelas. Metode ini membuat komunikasi antar proses lebih teratur dan aman.

1.4.TUGAS AKHIR

1. Tugas 1

a. Codingan

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void sorting_task() {
    int arr[5] = {5, 2, 9, 1, 6};
    for (int i = 0; i < 5; i++) {
        for (int j = i + 1; j < 5; j++) {
            if (arr[i] > arr[j]) {

```

```

        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

printf("[PID %d] Sorting selesai: ", getpid());
for (int i = 0; i < 5; i++) printf("%d ", arr[i]);
printf("\n");
}

void searching_task() {
    int arr[5] = {3, 8, 5, 2, 7}, key = 5;
    for (int i = 0; i < 5; i++) {
        if (arr[i] == key) {
            printf("[PID %d] Searching: angka %d ditemukan di indeks %d\n",
getpid(), key, i);
            return;
        }
    }
    printf("[PID %d] Searching: angka %d tidak ditemukan\n", getpid(),
key);
}

void calculation_task() {
    int sum = 0;
    for (int i = 1; i <= 10; i++) sum += i;
    printf("[PID %d] Calculation selesai. Jumlah = %d\n", getpid(), sum);
}

int main() {
    clock_t start, end;
    start = clock();

    pid_t pid1 = fork();
    if (pid1 == 0) {
        sorting_task();
        exit(0);
    }
}

```



```

pid_t pid2 = fork();
if (pid2 == 0) {
    searching_task();
    exit(0);
}

pid_t pid3 = fork();
if (pid3 == 0) {
    calculation_task();
    exit(0);
}

for (int i = 0; i < 3; i++) wait(NULL);

end = clock();
double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nParent PID: %d\n", getpid());
printf("Semua proses selesai.\n");
printf("Waktu eksekusi total: %.2f detik\n", time_taken);
return 0;
}

```

b. Penjelasan algoritma

1. Mulai Program

Program diawali dengan deklarasi fungsi-fungsi tugas:

sorting_task() untuk mengurutkan data

searching_task() untuk mencari nilai dalam array

calculation_task() untuk melakukan perhitungan jumlah angka 1–10

2. Mengukur Waktu Eksekusi

Program mencatat waktu mulai dengan variabel start menggunakan fungsi clock().

Ini digunakan untuk menghitung total waktu eksekusi semua proses.

3. Membuat Proses Anak Pertama (Child 1)

Program memanggil fungsi fork() dan menyimpannya ke variabel pid1.

Jika nilai pid1 == 0, berarti program berada di proses anak pertama.

Proses anak pertama menjalankan fungsi sorting_task().

Fungsi ini mengurutkan array {5, 2, 9, 1, 6} menggunakan metode sederhana (bubble sort).

Hasil akhir ditampilkan di terminal dengan menuliskan urutan angka yang sudah tersortir.

4. Membuat Proses Anak Kedua (Child 2)

Program kembali memanggil `fork()` dan menyimpannya ke variabel `pid2`.

Jika `pid2 == 0`, berarti ini adalah proses anak kedua.

Proses ini menjalankan fungsi `searching_task()`.

Fungsi akan mencari angka 5 di dalam array `{3, 8, 5, 2, 7}`.

Jika angka ditemukan, akan ditampilkan posisi indeksinya. Jika tidak ditemukan, akan ditampilkan pesan tidak ditemukan.

5. Membuat Proses Anak Ketiga (Child 3)

Program memanggil `fork()` lagi dan menyimpannya ke variabel `pid3`.

Jika `pid3 == 0`, maka proses ini menjalankan fungsi `calculation_task()`.

Fungsi ini menghitung jumlah total dari angka 1 sampai 10.

Hasil perhitungan ditampilkan di layar.

6. Menunggu Semua Proses Anak Selesai

Setelah membuat tiga proses anak, proses induk (parent process)

menggunakan fungsi `wait(NULL)` sebanyak tiga kali dalam perulangan.

Tujuannya agar proses induk menunggu sampai semua proses anak selesai dijalankan sebelum melanjutkan ke tahap berikutnya.

7. Menghitung dan Menampilkan Waktu Eksekusi Total

Setelah semua proses anak selesai, program mencatat waktu akhir dengan variabel `end` menggunakan fungsi `clock()`.

Waktu total eksekusi dihitung menggunakan rumus: $\text{time_taken} = (\text{end} - \text{start}) / \text{CLOCKS_PER_SEC}$

Hasil waktu total ditampilkan bersama PID dari proses induk.

8. Program Selesai

Program menampilkan informasi:

PID proses induk

Pesan bahwa semua proses telah selesai

Lama waktu eksekusi seluruh proses

Setelah itu, program berhenti dengan `return 0`.

c. Hasil output program

```

madha_27@DESKTOP-S58N8I3:~$ nano tugas1_process.c
madha_27@DESKTOP-S58N8I3:~$ gcc tugas1_process.c -o tugas1_process
madha_27@DESKTOP-S58N8I3:~$ ./tugas1_process
[PID 995] Sorting selesai: 1 2 5 6 9
[PID 996] Searching: angka 5 ditemukan di indeks 2
[PID 997] Calculation selesai. Jumlah = 55

Parent PID: 994
Semua proses selesai.
Waktu eksekusi total: 0.00 detik
madha_27@DESKTOP-S58N8I3:~$ |

```

2. Tugas 2

a. Codingan

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

int main() {
    char pesan[100];
    char nama_fifo1[] = "chat_user1";
    char nama_fifo2[] = "chat_user2";
    int fd_read, fd_write;
    char mode;

    printf("Pilih mode (1=User 1, 2=User 2): ");
    scanf(" %c", &mode);
    getchar(); // menghapus newline

    // Buat FIFO (named pipe) jika belum ada
    mkfifo(nama_fifo1, 0666);
    mkfifo(nama_fifo2, 0666);

    if (mode == '1') {
        printf("=== MODE: USER 1 ===\nKetik 'exit' untuk keluar.\n");

        while (1) {
            fd_write = open(nama_fifo1, O_WRONLY);
            printf("User 1: ");
            fgets(pesan, sizeof(pesan), stdin);
            write(fd_write, pesan, strlen(pesan)+1);

```

```

        close(fd_write);

        if (strncmp(pesan, "exit", 4) == 0)
            break;

        fd_read = open(nama_fifo2, O_RDONLY);
        read(fd_read, pesan, sizeof(pesan));
        printf("User 2: %s", pesan);
        close(fd_read);
    }
}
else if (mode == '2') {
    printf("=== MODE: USER 2 ===\nKetik 'exit' untuk keluar.\n");

    while (1) {
        fd_read = open(nama_fifo1, O_RDONLY);
        read(fd_read, pesan, sizeof(pesan));
        printf("User 1: %s", pesan);
        close(fd_read);

        if (strncmp(pesan, "exit", 4) == 0)
            break;

        fd_write = open(nama_fifo2, O_WRONLY);
        printf("User 2: ");
        fgets(pesan, sizeof(pesan), stdin);
        write(fd_write, pesan, strlen(pesan)+1);
        close(fd_write);

        if (strncmp(pesan, "exit", 4) == 0)
            break;
    }
}
else {
    printf("Mode tidak valid!\n");
}

unlink(nama_fifo1);
unlink(nama_fifo2);
return 0;

```

}

b. Penjelasan algoritma

1. Mulai Program

Mendeklarasikan variabel:

pesan[100] untuk menyimpan pesan yang dikirim.

nama_fifo1 dan nama_fifo2 sebagai nama FIFO untuk jalur komunikasi.

fd_read dan fd_write sebagai file descriptor.

mode untuk menentukan peran (User 1 atau User 2).

2. Input Mode User

Program menampilkan:

Pilih mode (1=User 1, 2=User 2):

Pengguna memasukkan angka:

1 berarti Mode User 1

2 berarti Mode User 2

3. Membuat FIFO (Named Pipe)

Program membuat dua FIFO dengan perintah:

```
mkfifo(nama_fifo1, 0666);
```

```
mkfifo(nama_fifo2, 0666);
```

Jika file chat_user1 dan chat_user2 belum ada, maka otomatis dibuat dengan izin baca dan tulis penuh.

4. Mode 1 (User 1)

Program menampilkan:

```
=== MODE: USER 1 ===
```

Ketik 'exit' untuk keluar.

Program masuk ke perulangan while(1) agar komunikasi bisa berlangsung terus-menerus.

User 1 menulis pesan ke FIFO chat_user1 menggunakan fungsi open() dan write().

Jika pesan adalah "exit", maka perulangan berhenti.

Jika belum, User 1 membaca balasan dari User 2 melalui FIFO chat_user2 menggunakan open() dan read().

5. Mode 2 (User 2)

Program menampilkan:

```
=== MODE: USER 2 ===
```

Ketik 'exit' untuk keluar.

Program masuk ke perulangan while(1).

User 2 membaca pesan dari chat_user1.
Jika pesan adalah “exit”, maka program berhenti.
Jika belum, User 2 menulis balasan ke chat_user2.

6. Menghapus FIFO Setelah Program Selesai

Setelah keluar dari perulangan, program menghapus file FIFO dengan perintah:

```
unlink(nama_fifo1);
```

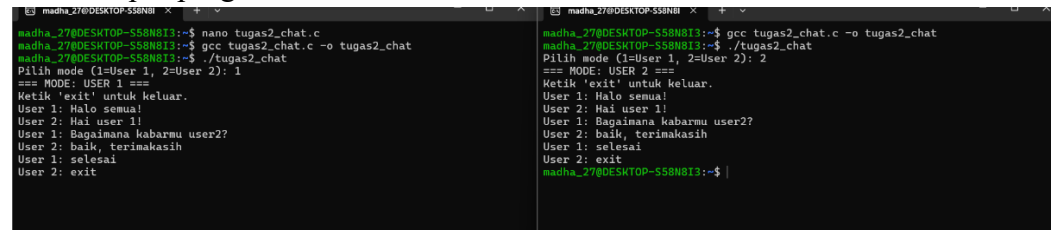
```
unlink(nama_fifo2);
```

Hal ini dilakukan agar file sementara tidak tersisa di direktori kerja.

7. Program Selesai

Program berhenti dan kembali ke sistem operasi dengan return 0.

c. Hasil output program



```
madha_27@DESKTOP-SS8N813:~$ nano tugas2_chat.c
madha_27@DESKTOP-SS8N813:~$ gcc tugas2_chat.c -o tugas2_chat
madha_27@DESKTOP-SS8N813:~$ ./tugas2_chat
Pilih mode (1=User 1, 2=User 2): 1
=== MODE: USER 1 ===
Ketik 'exit' untuk keluar.
User 1: Halo semua!
User 2: Hai user 1!
User 1: Bagaimana kabarmu user2?
User 2: baik, terimakasih
User 1: selesai
User 2: exit

madha_27@DESKTOP-SS8N813:~$ gcc tugas2_chat.c -o tugas2_chat
madha_27@DESKTOP-SS8N813:~$ ./tugas2_chat
Pilih mode (1=User 1, 2=User 2): 2
=== MODE: USER 2 ===
Ketik 'exit' untuk keluar.
User 1: Halo semua!
User 2: Hai user 1!
User 1: Bagaimana kabarmu user2?
User 2: baik, terimakasih
User 1: selesai
User 2: exit
madha_27@DESKTOP-SS8N813:~$
```

1.5.KESIMPULAN

Dari semua percobaan 1.1 sampai 2.4 dan tugas 1 serta 2, bisa disimpulkan bahwa kita belajar bagaimana sistem operasi bekerja dalam menjalankan dan mengatur proses di dalam komputer.

Pada percobaan 1, kita mempelajari cara membuat proses baru dengan `fork()`, bagaimana proses induk dan anak berjalan, serta bagaimana satu program bisa diganti dengan program lain menggunakan `exec()`. Dari sini kita jadi tahu kalau setiap proses punya peran masing-masing dan bisa berjalan bersamaan di sistem operasi.

Sedangkan pada percobaan 2, kita belajar bagaimana dua proses bisa saling berkomunikasi. Kita menggunakan beberapa cara seperti pipe, FIFO, shared memory, dan message queue. Dengan percobaan ini, kita tahu bahwa proses yang berbeda bisa saling mengirim dan menerima data, bahkan dari dua terminal yang berbeda.

Tugas 1 dan 2 membantu kita memahami dan mempraktikkan hal-hal tersebut secara langsung di Ubuntu. Kita belajar membuat program, menjalankannya, melihat hasilnya, dan memahami bagaimana proses saling bekerja sama.

Secara keseluruhan, percobaan dan tugas ini membuat kita lebih paham tentang cara kerja proses dan komunikasi antar proses di sistem operasi, serta bagaimana program bisa berjalan bersama dan saling bertukar informasi dengan baik.